



Data Extraction Project Report

Internships Scrapping

Ellington Kirby - Lucas Fourest

Master IASD - Université PSL

January 7, 2024

Contents

1	Overview	1
2	Data sources	1
2.1	INRIA	1
2.2	DIIP	2
2.3	IASD	2
3	Data extraction and restructuring: tools and methods	3
3.1	INRIA	3
3.2	DIIP	4
3.3	IASD	4
4	Data storage	5
5	Quality assessment and additional functionalities	6
5.1	Quality assessment	6
5.2	Inverted index for keyword search	6

1 Overview

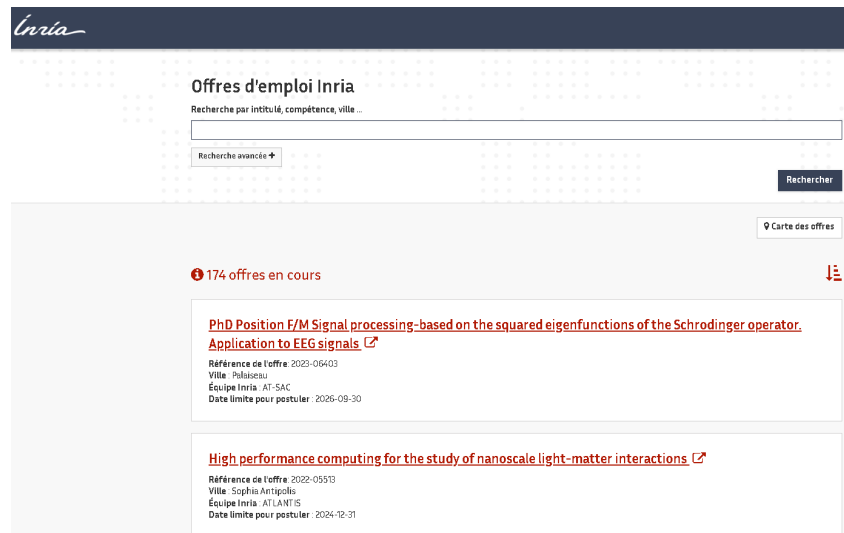
In this report we summarize the overall work that led to the constitution of our final database from A to Z. The goal of this database is to gather a certain amount of internships opened propositions, with a few descriptive attributes (organization, supervisor, mission ...), extracted from research and/or academic institutions webpages, namely INRIA, DIIP and IASD webpages. We will make sure to clearly explain our extraction methods, storage tools, database functionalities as well as all the key choices that were made.

2 Data sources

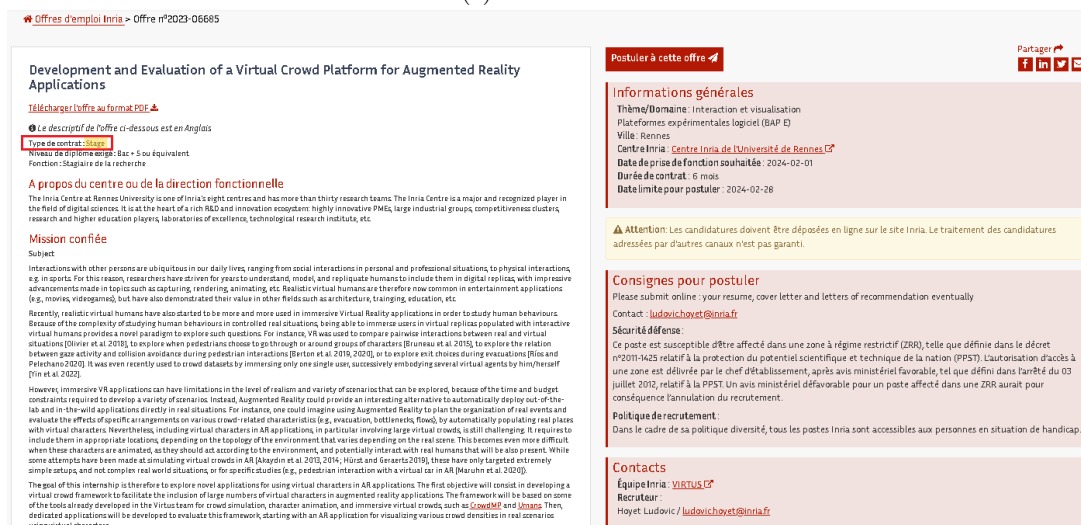
To begin with, we will briefly present each of the different websites we relied on, e.g. their nature and the structure of the content of interest.

2.1 INRIA

The first data source we consider is the recruiting website of the (French) National Institute for Research in Artificial Intelligence (INRIA), which gather several opened opportunities of different kind (PhD, permanent contract...) in an "offer" section.



(a) Offer section



(b) Example of an offer page, which category is "stage"

When clicked, the hyperlink of an opportunity redirects toward another page that displays all the available information about the corresponding job, including its category. Our category of interest for this work will be the "stage" one, and our starting page the "offer" section.

2.2 DIIP

The second data source we considered is the website of the Data Intelligence Institute of Paris (DIIP). The "News" section contains a reference toward another page (of the same website) with recent master internship opportunities for the current year.

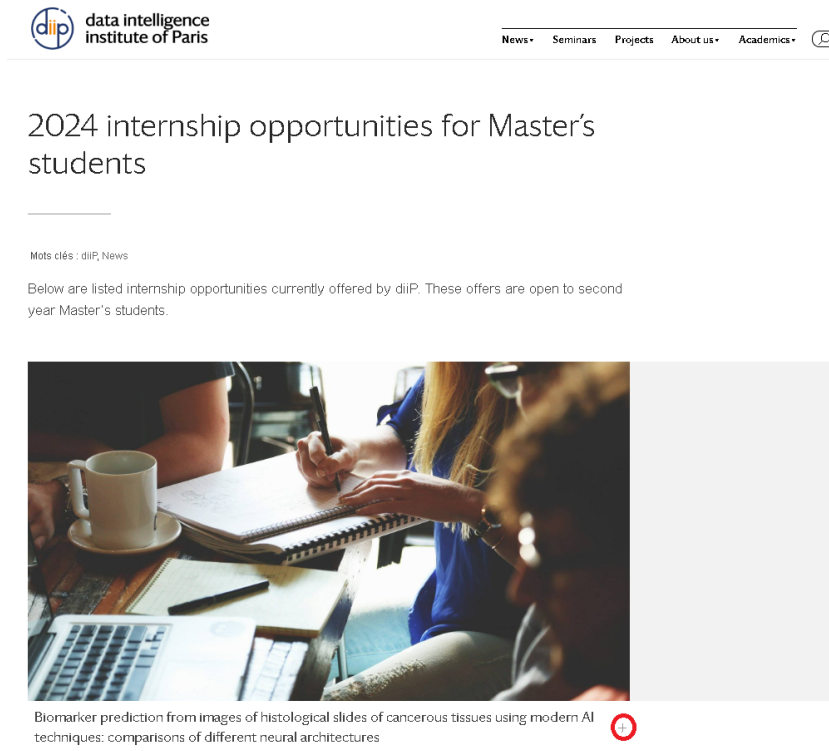


Figure 2: Internship section

This will be our starting point for this source. On this page we can directly find all the internship propositions as well as their relevant information simply by developing the "+" symbol besides their name, hence all the content we need is available in this single page without redirection.

2.3 IASD

Our third and last data source is the master website. This time, the website simply gather all the internship opportunities in a single "Internship" division, hence it will be our starting page.

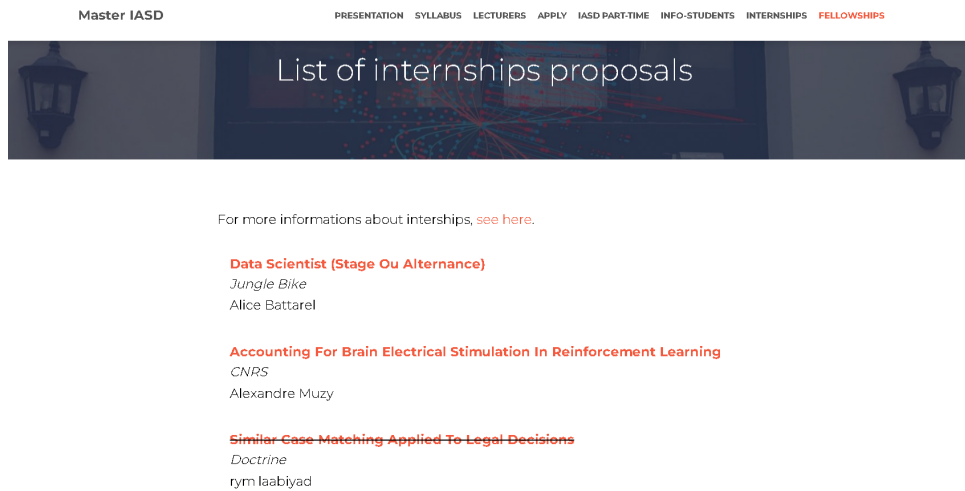


Figure 3: Internship section

Nevertheless, this site is more specific in that, instead of redirecting to another page, the internships hyperlinks trigger the download of a pdf file containing a full description. As a consequence, some elements of information are directly available on the page and other not. The way pdf files are processed to obtain all the final attributes will be further discussed. It is also noteworthy that an internship opportunity present on the page might not be available anymore: in that case the corresponding title is crossed out.

3 Data extraction and restructuring: tools and methods

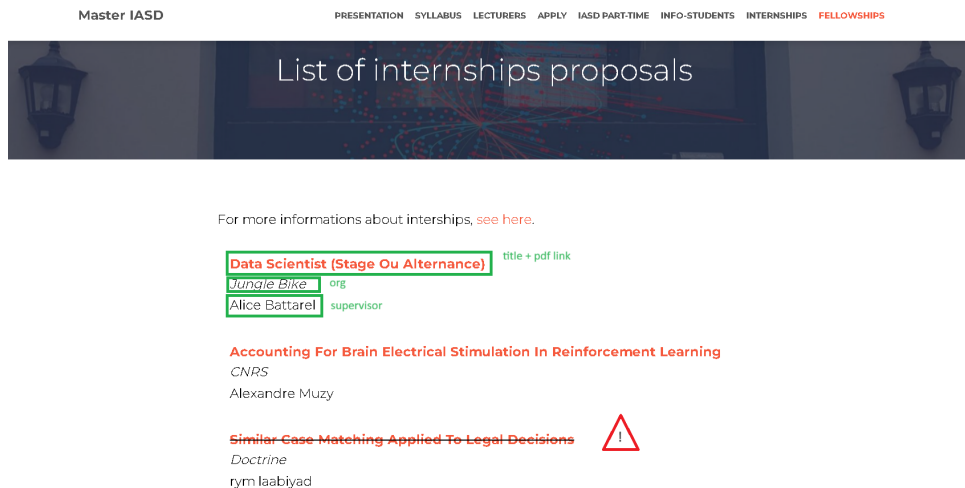
In this section we briefly present the core ideas that were used to collect and structure the data from each website. As we consider webpages and HTML content, we used scrapping with the Scrapy python library tool to collect the useful data about available internships across the diverse websites. Because we are only dealing with three different sources, we made the choice to design three “small” different scrappers rather than a single one able to “generalize” the scrapping process to each source. The resulting json files share a common structure:

```
{ 'title': title,
  'organization': organization,
  'supervisor': supervisor,
  'description': internship description,
  'link': pdf link }
```

To be noted that each of the mentioned sources provides a pdf link for their forwarded offers. We will further detail the core ideas we used to extract data from each of the websites with their own specificities, in order to clarify the method behind our code.

3.1 INRIA

As explained 2.1, we provide the “offer” section as a starting url for our INRIA scrapper `inria_spider.py`, which regroupes all the redirection links toward all the available offers from any category.



The second and main challenge is that there is no direct description available somewhere on the web: the actual description is contained in the pdf file associated to the title hyperlink. To overcome this limitation, we relied on the LangChain library and OpenAI's API to carry out *Retrieved Augmented Generation* (RAG) with as input the pdf link. This restructuring method consists in writing a concise summary of the pdf content in N words (here we chose $N = 100$) using the power of Large Language Models (LLMs). In a dedicated file `internship_description_generator.py`, our script sequentially take the pdf links provided by the first obtained `iasd.json` file, use them to generate a description for each internship with the previously mentionned blackbox tools, and generates the complete and final json file for IASD `retrieved_iasd.json`.

4 Data storage

The extracted data regroups several internships with a common set of textual features (title, organization, supervisor, description...), therefore it seems natural to store it as relational data, in a table where the columns would be the so called textual features.

We deemed a Database Management System (DBMS) could prove useful for effective management of our internship relational data. On a first place it can ensure the imposition of any type of keys and constraints, maintaining overall data integrity. It is also designed to optimize query performance, which could be crucial for scalability if our internships database came to be significantly large. Especially, we chose **SQLite** as a DBMS, for several reasons:

- Portability: SQLite databases are stored in a simple file, making them easily portable. This can be advantageous if we need to move or share the database.
- Good performance for moderate workloads: SQLite is well-suited for applications with moderate performance requirements. For an internship proposals database, where the workload may not be very heavy, SQLite offers adequate performance without introducing unnecessary complexity.

Hence we used SQLite to constitute a tabular database with a single table `internship` having the following relational schema:

```
TABLE internship(title TEXT UNIQUE,
organization TEXT,
supervisor TEXT,
description TEXT,
source TEXT,
PRIMARY KEY (title,organization))
```

Where source stands for the related institution name (IASD, DIIP or INRIA) and where we choose the pair of features (title, organization) as a (non-atomic) primary key: two internships with the same title from the same organization are very likely to be the same offer. The insertion mode `INSERT OR IGNORE` ignores new entries with already taken or invalid primary key. This method avoid eventual doublons of

a same internship scrapped from different websites. We can also note that we got rid of the 'link' key present in each json file: we deemed this attribute as useless in our final database.

5 Quality assessment and additional functionalities

We finished this work by designing a couple of methods to assess the "quality" of our database and by adding a keyword search for further use. Indeed, such a functionality is naturally suited to our database, allowing future users to search more or less specific internship topics in a restricted domain.

5.1 Quality assessment

After clarifying a few specific points that could illustrate the quality of data in our case, we decided to set up the following "quality tests":

- Completeness : how many NULL values are present in each column?
- Redundancy: How many duplicates are present in the database?
- Diversity: For each different origin source, how many internships are in the database ?
- Verbosity: What are the lengths of internships descriptions and their distribution (in terms of length)?

To be noted that the redundancy test should be useless if our primary key 4 is well designed, still we double check as we do not have this guarantee.

5.2 Inverted index for keyword search

Finally, we decided to set up an `inverted_search_index.py` script for keyword search. As explained previously, this functionality comes well with an internships database, as it would allow student users for a finer search that would better corresponds their needs. We modeled it off a map reduce style script so it could be adapted in the future.

Using the Term Frequency-Inverse Document Frequency (TF-IDF) method, our functionality can assign a relevance score to each internship in the database regarding a specific keyword. It does not replace an exhaustive search of the student, but such a ranking could reorder the propositions in a way that helps the user for a faster search, with supposedly top propositions in the first place.